NetFront ESP32 Hardware and Daktronics Parser Reference

Scoreboard bridge pinout, power, interfaces, and All Sport CG CONTROL feed mapping

Serial Configuration

Setting	Value
Purpose	Read clock, running state, scores, shots, period, and penalties
Source port	Daktronics All Sport CG CONTROL port (DB-9)
Electrical interface	RS-232 through MAX3232 level converter; do not connect DB-9 RS-232 directly to ESP GPIO
UART	`9600 baud, 8 data bits, no parity, 1 stop bit (8N1)`
Flow control	None
ESP32 pins	RX GPIO16, TX GPIO17

ESP32 Hardware Pin Inventory

ESP32 pin	Direction / mode	Current function	Status
`GPIO0 / BOOT`	Input with internal pull-up; active LOW	Hold BOOT for 7 seconds while firmware is running to clear NetFront preferences and Wi-Fi credentials, then restart. Holding BOOT during reset enters the ESP32 ROM bootloader instead.	Explicit firmware configuration
`GPIO1 / U0TXD`	UART0 transmit	USB serial console output, boot messages, logs, and PlatformIO monitoring at 115200 baud.	ESP32 Dev Module default used by `Serial`
`GPIO3 / U0RXD`	UART0 receive	USB serial console input and firmware upload/debug connection through the development board USB-to-serial interface.	ESP32 Dev Module default used by `Serial`
`GPIO16 / UART1 RX`	Input, 3.3V TTL UART	Receives the level-shifted Daktronics All Sport CG serial stream from the MAX3232 receiver output.	Explicit firmware configuration
`GPIO17 / UART1 TX`	Output, 3.3V TTL UART	UART1 transmit path toward the MAX3232 transmitter input. The current scoreboard parser is receive-driven and does not normally send CG commands.	Explicit firmware configuration
`EN / RESET`	Hardware reset input; active LOW	Restarts the ESP32 immediately. It does not erase settings or perform a factory reset.	ESP32 development board hardware
Other GPIO	Not configured by this application	No application status LEDs, relays, or additional buttons are currently assigned.	Confirmed by firmware inventory

Power and Electrical Requirements

Connection	Requirement	Notes
ESP32 power	Use the development board USB connector or a regulated supply appropriate for its labeled `5V/VIN` input.	The firmware cannot identify the physical supply source. Do not apply an unregulated source or exceed the specific board manufacturer's rating.
ESP32 GPIO logic	**3.3V only**	GPIO16 and GPIO17 are not RS-232 tolerant and are not 5V tolerant. Never connect the Daktronics DB-9 directly to either GPIO.
MAX3232 power	Power the module at a voltage supported by that exact module.	The MAX3232 IC commonly supports 3.0-5.5V, but breakout modules vary. Verify the module label/datasheet. Its TTL output to GPIO16 must not exceed 3.3V.
Ground	ESP32 and MAX3232 TTL-side grounds must be common.	The DB-9 signal ground must also be connected according to the confirmed Daktronics CONTROL-port cable/pinout.
Wi-Fi	2.4 GHz ESP32 Wi-Fi	Firmware disables Wi-Fi power saving for stable bridge communication and retries station reconnection every 10 seconds.

> **Power information requiring physical verification:** the repository does not record the exact ESP32 board revision, MAX3232 breakout model, its VCC connection, or the installed DB-9 conductor pinout. Confirm those items on the assembled device before using this document as a fabrication drawing.

Scoreboard Interface Wiring

Signal path	Connection	Function
Daktronics transmit	CG CONTROL-port TX conductor → MAX3232 RS-232 receiver input	Accepts the positive/negative RS-232 waveform from the scoreboard device.
Bridge receive	MAX3232 receiver TTL output → ESP32 `GPIO16`	Delivers the converted 3.3V-compatible 9600 8N1 stream to UART1 RX.
Bridge transmit	ESP32 `GPIO17` → MAX3232 transmitter TTL input	Provides a return transmit path if required later; current production parsing is receive-only.
Daktronics receive	MAX3232 RS-232 transmitter output → CG CONTROL-port RX conductor	Optional return path; physical DB-9 assignment must be confirmed.
Common	ESP32 GND ↔ MAX3232 GND; DB-9 signal common per confirmed cable	Establishes the required voltage reference.

```
Daktronics CG DB-9 (RS-232)          MAX3232                 ESP32 Dev Module
  TX  ----------------------------> R1IN  |  R1OUT  --------> GPIO16 (UART1 RX)
  RX  <---------------------------- T1OUT |  T1IN   <-------- GPIO17 (UART1 TX)
  Signal common ---------------------- GND  -------------------  GND
```

> **DB-9 pin numbers are intentionally not asserted.** DTE/DCE orientation can swap pins 2 and 3, and the repository does not prove the installed cable pinout. Verify CG TX, CG RX, and signal common from the Daktronics documentation or with a meter before wiring.

Boot, Reset, and Upload Behavior

Action	Result
Press `EN`	Immediate hardware reboot; settings remain stored.
Hold `BOOT` for at least 7 seconds while running	Firmware factory reset: clears the `netfront` NVS namespace and cached Wi-Fi credentials, then reboots into default AP mode.
Hold `BOOT`, press/release `EN`	Enters ROM download/flash mode; this does not execute the application factory-reset routine.
PlatformIO upload	USB UART upload at a configured speed of 921600 baud.
PlatformIO serial monitor	UART0 console at 115200 baud.

Board, Flash, and Storage Configuration

Item	Current configuration
PlatformIO board	`esp32dev` using the Arduino framework
Filesystem	LittleFS; stores the Admin, Simulator, Serial Analyzer, WebSocket Diagnostics, and Debug JSON pages
Persistent settings	ESP32 NVS/Preferences namespace `netfront`: admin password, WebSocket token, SSID/password, DHCP/static IP, gateway, and subnet
Flash / partitions	PlatformIO `esp32dev` defaults; no custom partition table is declared
PSRAM	Not enabled in the project configuration
OTA upload	Not configured; current firmware deployment uses USB serial

Frame Structure

```
<01>[fixed-width payload]<04>
```

Element	Hex	Meaning
Start marker	`0x01`	SOH; resets frame capture and starts a new payload
End marker	`0x04`	EOT; completes the frame and invokes parsing

All column numbers below are zero-based offsets within the payload, excluding SOH and EOT.

Main Mapped Fields

Columns	Width	Mapped field	Parser behavior
`0-4`	5	Game clock	`MM:SS`; minutes at 0-1, colon at 2, seconds at 3-4
`5-6`	2	Reserved / spacing	Ignored
`7`	1	Clock state	`s` or `S` = stopped; blank = running
`8-9`	2	Home score	Integer, clamped to 0-199
`10-11`	2	Away score	Integer, clamped to 0-199
`12`	1	Reserved / spacing	Ignored
`13-15`	3	Home shots	Integer, clamped to 0-199
`16-17`	2	Away shots	Integer, clamped to 0-199
`18`	1	Period	Integer, clamped to 1-9
`19`	1	Separator	Ignored

Penalty Mapped Fields

Each penalty slot is exactly seven characters with the format `PP M:SS`.

Columns	Mapped slot	Example player	Example remaining time
`20-26`	Home Penalty 1	25	1:34
`27-33`	Home Penalty 2	36	1:38
`34-40`	Away Penalty 1	54	1:45
`41-47`	Away Penalty 2	55	1:50

Penalty Slot Subfields

Relative columns	Meaning	Validation
`0-1`	Player number	Must contain a player number greater than zero
`2`	Separator	Normally a space
`3`	Minutes remaining	Single digit in current observed format
`4`	Colon	Must be `:`
`5-6`	Seconds remaining	Must be 00-59

> A valid slot with remaining time above `0:00` is active. A blank, malformed, or zero-time slot is cleared/inactive. The CG feed remains authoritative for penalty time in hardware mode.

Observed Example

```
<01>16:33  s 2 211    1 25 1:3436 1:3854 1:4555 1:500<04>
```

Mapped value	Decoded result
Clock	16:33, stopped
Home Penalty 1	Player 25, 1:34 remaining
Home Penalty 2	Player 36, 1:38 remaining
Away Penalty 1	Player 54, 1:45 remaining
Away Penalty 2	Player 55, 1:50 remaining

Unknown trailing field: payload data at column `48+` has included values such as `0` and `16`. It is likely a control or checksum field and is intentionally ignored until its definition is confirmed.

ESP Output Fields

The full state and WebSocket payload expose these mapped properties:

- `clock`
- `clockFormatted`
- `clockRunning`
- `clockMin`
- `clockSec`
- `clockTenths`
- `homeScore`
- `awayScore`
- `homeShots`
- `awayShots`
- `period`
- `homePen1Player`
- `homePen1Time`
- `homePen2Player`
- `homePen2Time`
- `awayPen1Player`
- `awayPen1Time`
- `awayPen2Player`
- `awayPen2Time`

Serial Analyzer Penalty Array

```
{
  "team": 0,
  "slot": 1,
  "player": 25,
  "minutes": 1,
  "seconds": 34
}
```

Value	Meaning
`team: 0`	Home
`team: 1`	Away
`slot: 1`	First penalty slot for that team
`slot: 2`	Second penalty slot for that team

Parser Behavior

- UART bytes are captured between SOH and EOT into a fixed-capacity buffer.
- Native production frames use fixed column offsets and do not rely on whitespace tokenization.
- Incomplete or malformed penalty slots are cleared rather than retaining stale values.
- Clock tenths are set to zero because the observed CG hardware feed provides whole seconds only.
- The unknown trailing control/checksum field is not validated.
- The Serial Analyzer presents escaped control bytes such as `<01>` and `<04>`.

Text Test-Feed Fallback

The firmware retains a line-oriented key/value parser for simulator and diagnostic feeds. Supported keys are:

Mapped value	Accepted keys
Clock	`CLK` , `CLOCK` , `TIME`
Period	`PER` , `PERIOD`
Home score	`HOME` , `HOM` , `HSCORE`
Away score	`AWAY` , `AWY` , `ASCORE`
Home shots	`HSHOTS` , `SHOTH` , `HSH`
Away shots	`ASHOTS` , `SHOTA` , `ASH`
Clock running	`RUN` , `CLOCKRUN` , `RUNNING`
Penalty slots	`HP1` , `HP2` , `AP1` , `AP2`

Relevant ESP Endpoints

Endpoint	Purpose
`/api/state`	Current full gateway state
`/api/state/clock`	Clock and period payload
`/api/state/full`	Explicit full scoreboard payload
`/serial.json`	Legacy serial state, penalty array, RX diagnostics, and raw trace
`/admin.json`	Admin/debug representation of full state
`/ws`	Authenticated WebSocket state stream